

Soft Gradient Boosting With Learnable Feature Transforms for Sequential Regression

Huseyin Karaca  and Suleyman Serdar Kozat , *Senior Member, IEEE*

Abstract—We propose a soft gradient boosting framework for sequential regression that embeds a learnable linear feature transform within the boosting procedure. At each boosting iteration, we train a soft decision tree and learn a linear input feature transform $\mathbf{Q}$ together. This approach is particularly advantageous in high-dimensional, data-scarce scenarios, as it discovers the most relevant input representations while boosting. We demonstrate, using both synthetic and real-world datasets, that our method effectively and efficiently increases the performance by an end-to-end optimization of feature selection/transform and boosting while avoiding overfitting. We also extend our algorithm to differentiable non-linear transforms if overfitting is not a problem. To support reproducibility and future work, we share our code publicly.

Index Terms—Gradient boosting, sequential regression, soft decision trees.

I. INTRODUCTION

IN THIS letter, we study sequential (online) regression problem, which aims to regress the next value y_{T+1} of a series $\{y_t\}_{t=1}^T$ (optionally conditioned on an exogenous features $\{\mathbf{x}_t\}_{t=1}^T$) given all information up to time T .¹ This problem models a wide range of applications, such as energy-load forecasting, IoT and traffic management [1], [2], and has been studied extensively in the signal processing literature.

Gradientboosted decisiontree models (e.g. XGBoost [3], LightGBM [4]) are the current state of the art as demonstrated in different real life competitions [5], [6]. Yet they struggle when the feature dimension is large relative to the amount of available training data—a frequent situation that occurs in practice (e.g. short sales histories paired with hundreds of engineered covariates) [7], [8]. Exhaustive subset or interaction searches of those covariates are usually restrictive, making them impractical in data-scarce, highdimensional scenarios [9], [10].

As a solution, we propose a boosting framework that overcomes this bottleneck by inserting a learnable linear feature transform between successive weak (base) learners in gradient-boosting. We concentrate on linear transforms to avoid overfitting; however, we also extend our algorithm to differentiable

nonlinear transforms, with or without memory (e.g., neural networks, RNNs, and similar), or to standard boosting machines using hard decision trees (e.g., vanilla LightGBM, XGBoost, CatBoost). Specifically, we

- 1) replace hard trees with soft (differentiable) decision trees, enabling end-to-end optimization,
- 2) learn a transform matrix $\mathbf{Q}$ to perform embedded feature selection/transformation at every boosting round, which is jointly optimized with the newly fitted tree,
- 3) demonstrate that this framework can be readily extended to nonlinear feature transformations,
- 4) openly share our code for replicability of our results and encourage further research.²

There are also existing studies in literature that combine soft decision trees with gradient boosting [11], and others integrating feature selection concepts into boosting methods [12], [13]. The method introduced in [12] rely only on features specific to certain applications such as HOG features used in computer vision, which limits its general applicability. The other related work [13] combines traditional gradient-boosted decision trees with feature selection. Their method works effectively when the dataset has many samples compared to the number of features. However, in many real-world problems, especially in financial forecasting, we may frequently face the opposite case: few data points relative to many features [14].

Hence, we introduce an end-to-end optimized method for model training, especially for situations with many features and limited data. Note that by this basic transformation, we avoid both overfitting and remedy the need for feature selection and/or transformation, where the nonlinear interactions are discovered by the boosted decision trees. We demonstrate through experiments—both with synthetic and real-world data—that our method significantly increases the performance of boosted trees while introducing little computational overhead in scenarios where data is scarce and feature space is large.

II. PROBLEM DEFINITION

A. Sequential Regression

We sequentially observe $\{y_t\}_{t \geq 1}$, $y_t \in \mathbb{R}$ along with the feature vectors $\{\mathbf{x}_t\}_{t \geq 1}$, $\mathbf{x}_t \in \mathbb{R}^d$. Our goal is to predict y_{t+1} at each t in an online manner by constructing a function that depends only on the past information:

$$\hat{y}_{t+1} = f(\{\dots, y_{t-1}, y_t\}, \{\dots, \mathbf{x}_{t-1}, \mathbf{x}_t\}).$$

At each time step t , we output $\hat{y}_{t+1}$ and suffer a loss $\mathcal{L}(y_{t+1}, \hat{y}_{t+1})$. The loss function $\mathcal{L}$ can be any differentiable

²<https://github.com/huseyin-karaca/bsdtg>

Received 15 October 2025; revised 16 November 2025; accepted 20 November 2025. Date of publication 2 December 2025; date of current version 26 December 2025. This work was supported by TÜBİTAK (The Scientific and Technological Research Council of Turkey). The associate editor coordinating the review of this article and approving it for publication was Prof. Haiquan Zhao. (Corresponding author: Huseyin Karaca.)

The authors are with the Department of Electrical and Electronics Engineering, Bilkent University, Ankara 06800, Türkiye (e-mail: huseyin.karaca@bilkent.edu.tr; kozat@ee.bilkent.edu.tr).

Digital Object Identifier 10.1109/LSP.2025.3639345

¹We denote all vectors by boldface lowercase letters (e.g. $\mathbf{u}$), and matrices by boldface capital letters (e.g. $\mathbf{U}$). For a vector $\mathbf{u}$ (or matrix $\mathbf{U}$), $\mathbf{u}^\top$ ($\mathbf{U}^\top$) is the transpose. The time index is indicated by a subscript; for instance, $\mathbf{u}_t$ is a vector at time t . $\mathbf{I}_d \in \mathbb{R}^{d \times d}$ is the d -dimensional identity matrix.

Algorithm 1: Boosted Soft Decision Trees with feature transform $\mathbf{Q}$ (BSDT-Q).

Input: $\mathbf{X}_{\text{train}} \in \mathbb{R}^{n \times d}$, targets $\mathbf{y} \in \mathbb{R}^n$, number of boosting rounds K
Output: $\{(\mathbf{Q}_k, \text{SDT}_k)\}_{k=1}^K$, where SDT_k is the base Soft Decision Tree at k 'th step, $\mathbf{Q}_k$ is the feature transform matrix
Initialize ensemble prediction $\hat{\mathbf{y}}^{(0)} \leftarrow \mathbf{0}$, linear transform $\mathbf{Q}_0 \leftarrow \mathbf{I}_d$;
for $k \leftarrow 1$ **to** K **do**
 Residual $\mathbf{r}_k \leftarrow \mathbf{y} - \hat{\mathbf{y}}_{k-1}$;
 Fit soft decision tree SDT_k on $(\mathbf{Q}_{k-1} \mathbf{X}_{\text{train}}, \mathbf{r}_k)$;
 Find $\mathbf{Q}_k$ (for a number of gradient-descent steps w.r.t. loss of SDT_k); $\hat{\mathbf{y}}^{(k)} \leftarrow \hat{\mathbf{y}}^{(k-1)} + \text{SDT}_k(\mathbf{Q}_k \mathbf{X}_{\text{train}})$;
end
return $\{(\mathbf{Q}_k, \text{SDT}_k)\}_{k=1}^K$

function such as the squared error loss $(y_{t+1} - \hat{y}_{t+1})^2$. We aim to minimize the accumulated online error over time: $\mathcal{L}_T = \sum_{t=1}^T \mathcal{L}(y_t, \hat{y}_t)$. Note that this main framework can be readily extended to different horizons, e.g., predicting $y_{t+2}, \dots, y_{t+n}$, either using direct or recursive approaches [15].

B. Hard Decision Trees

A hard decision tree (HDT) $f(\mathbf{z})$ can be written as a sum of leaf values multiplied by indicator functions $f(\mathbf{z}) = \sum_{n=1}^N \gamma_n \mathbf{1}(\mathbf{z} \in P_n)$, where P_n denotes the region (leaf) in the input space associated with the n -th leaf, and $\mathbf{1}(\cdot)$ is the indicator function that returns 1 if $\mathbf{z}$ belongs to P_n and 0 otherwise. Hard decision trees use indicator-based splits (e.g., if $z_j < \text{threshold}$, go left; else go right). These splits are not differentiable with respect to the input $\mathbf{z}$, hence there is no straightforward way to compute $\nabla_{\mathbf{Q}} f(\mathbf{Q}\mathbf{x})$. This makes gradient-based optimization of $\mathbf{Q}$ challenging. Nevertheless, in many real-world applications (including sequential regression), hard decision trees are crucial due to their interpretability and strong performance, especially when they are used in a gradient-boosting framework.

C. Soft Decision Trees

Soft decision trees (SDT) relax the hard splits of hard decision trees into differentiable “soft” gating functions, often using a logistic $\sigma(\cdot)$ at each node. Let ℓ index a leaf in the set of leaves ℓ , and let $\text{path}(\ell)$ denote the set of node indices along the path from the root to leaf ℓ . Then each node i outputs a probability $p_i(\mathbf{z}) = \sigma(\mathbf{w}_i^\top \mathbf{z} + b_i)$. Depending on whether the node “direction” $v_{i,\ell}$ is 1 (go right) or 0 (go left), the contribution to the final leaf value is $p_i(\mathbf{z})$ or $1 - p_i(\mathbf{z})$, respectively. Thus, the probability that a given input $\mathbf{z}$ is routed to a particular leaf ℓ , denoted as p_ℓ^* , is calculated by multiplying the soft gating decisions along the entire path from the root to the leaf:

$$p_\ell^* = \prod_{i \in \text{path}(\ell)} [p_i(\mathbf{z})^{v_{i,\ell}} (1 - p_i(\mathbf{z}))^{1-v_{i,\ell}}]. \quad (1)$$

The overall tree output is then $f(\mathbf{z}) = \sum_{\ell \in \ell} p_\ell^*(\mathbf{z}) \gamma_\ell$, where γ_ℓ is the value assigned to the leaf ℓ .³ Due to $\sigma(\cdot)$, $f(\mathbf{z})$ is

³For the shallow tree depths typically used in boosting (e.g., $d \leq 6$), the product of path probabilities is numerically stable. However, for deeper trees,

differentiable with respect to $\mathbf{z}$, and hence with respect to $\mathbf{Q}$. We calculate a gradient-descent update to $\mathbf{Q}$, using differentiability of soft trees in Section III.

D. Gradient Boosting Machines

Gradient Boosting Machines [16] build a strong predictor by stage-wise accumulation of weak (base) learners. Given training data $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ and a differentiable loss $\mathcal{L}(y, F(\mathbf{x}))$, the ensemble after m rounds is given by $F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \nu f_m(\mathbf{x})$, $0 < \nu \leq 1$, where f_m is the new weak learner and ν is a shrinkage factor. At each round m we learn the new weak learner f_m to be added to the ensemble as

$$f_m = \arg \min_{f \in \mathcal{H}} \sum_{i=1}^N (g_{i,m} - f(\mathbf{x}_i))^2 + \Omega(f)$$

where $\mathcal{H}$ is the hypothesis space (e.g. hard decision trees), Ω is the regularizer. The $g_{i,m}$ represent the negative gradient of the loss function (with respect to the function values of the current ensemble):

$$g_{i,m} = - \frac{\partial \mathcal{L}(y_i, F)}{\partial F} \Big|_{F=F_{m-1}(\mathbf{x}_i)} \quad i = 1, \dots, N.$$

Thus, at each iteration we fit a new weak learner to the residual errors so that it focuses on the observations the ensemble currently predicts poorly.

III. LEARNING AN EFFICIENT FEATURE TRANSFORM FOR SOFT DECISION TREES

When using a soft decision tree f and an input transform $\mathbf{z} = \mathbf{Q}\mathbf{x}$, where $\mathbf{x} \in \mathbb{R}^{d_{in}}$, $\mathbf{z} \in \mathbb{R}^{d_{out}}$, $\mathbf{Q} \in \mathbb{R}^{d_{out} \times d_{in}}$, we can perform gradient-based updates of $\mathbf{Q}$ as

$$\mathbf{Q}_{k+1} \leftarrow \mathbf{Q}_k - \eta \nabla_{\mathbf{Q}_k} [\mathcal{L}(y, f(\mathbf{Q}_k \mathbf{x}))],$$

where η is the learning rate. Following the definitions given in Section II-C and rewriting (1), we have

$$p_\ell^*(\mathbf{z}) = \prod_{i \in \text{path}(\ell)} \beta_{i,\ell}(\mathbf{z}),$$

where

$$\beta_{i,\ell}(\mathbf{z}) = \begin{cases} p_i(\mathbf{z}), & v_{i,\ell} = 1, \\ 1 - p_i(\mathbf{z}), & v_{i,\ell} = 0, \end{cases}$$

and $p_i(\mathbf{z}) = \sigma(\mathbf{w}_i^\top \mathbf{z} + b_i)$. By the product rule,

$$\frac{\partial p_\ell^*(\mathbf{z})}{\partial z_k} = p_\ell^*(\mathbf{z}) \sum_{m \in \text{path}(\ell)} \frac{1}{\beta_{m,\ell}(\mathbf{z})} \frac{\partial \beta_{m,\ell}(\mathbf{z})}{\partial z_k}.$$

Since $\beta_{m,\ell}(\mathbf{z})$ is either $p_m(\mathbf{z})$ or $1 - p_m(\mathbf{z})$, we get $\frac{1}{\beta_{m,\ell}(\mathbf{z})} \frac{\partial \beta_{m,\ell}(\mathbf{z})}{\partial z_k} = (v_{m,\ell} - p_m(\mathbf{z})) w_{m,k}$, where $\mathbf{w}_m = (w_{m,1}, \dots, w_{m,d_{out}})^\top$ is the node's parameter vector. Summing over m yields

$$\frac{\partial p_\ell^*(\mathbf{z})}{\partial z_k} = p_\ell^*(\mathbf{z}) \sum_{m \in \text{path}(\ell)} [v_{m,\ell} - p_m(\mathbf{z})] w_{m,k}.$$

numerical stability can be further enhanced by working with log-probabilities or by applying probability clipping to prevent values from approaching zero.

We can now formulate the partial derivative of the tree's output $f(\mathbf{z})$ by summing the contributions from each leaf, which is a necessary step for backpropagation,

$$\begin{aligned} \frac{\partial f(\mathbf{z})}{\partial z_k} &= \sum_{\ell \in \mathcal{L}} \gamma_\ell \frac{\partial p_\ell^*(\mathbf{z})}{\partial z_k} \\ &= \sum_{\ell \in \mathcal{L}} \gamma_\ell p_\ell^*(\mathbf{z}) \sum_{m \in \text{path}(\ell)} [v_{m,\ell} - p_m(\mathbf{z})] w_{m,k}. \end{aligned} \quad (2)$$

Let $\mathbf{z} = \mathbf{Q}\mathbf{x}$. Then for each entry $Q_{r,c}$,

$$\frac{\partial f(\mathbf{Q}\mathbf{x})}{\partial Q_{r,c}} = \sum_{k=1}^{d_{\text{out}}} \frac{\partial f(\mathbf{z})}{\partial z_k} \frac{\partial z_k}{\partial Q_{r,c}}. \quad (3)$$

However, $z_k = \sum_{j=1}^{d_{\text{in}}} Q_{k,j} x_j$, thus $\partial z_k / \partial Q_{r,c} = x_c$ if $k = r$ and 0 otherwise, making $\frac{\partial f(\mathbf{Q}\mathbf{x})}{\partial Q_{r,c}} = \left(\frac{\partial f(\mathbf{z})}{\partial z_r}\right) x_c$. Finally, the elementwise result can be obtained as

$$\frac{\partial f(\mathbf{Q}\mathbf{x})}{\partial Q_{r,c}} = x_c \sum_{\ell} \gamma_\ell p_\ell^*(\mathbf{z}) \sum_{m \in \text{path}(\ell)} [v_{m,\ell} - p_m(\mathbf{z})] w_{m,r}.$$

Putting it all together yields the final expression for $\nabla_{\mathbf{Q}} f(\mathbf{Q}\mathbf{x})$. Then we can combine it with the loss $\mathcal{L}(y, f(\mathbf{Q}\mathbf{x}))$ to update $\mathbf{Q}$ via gradient descent. For example, in the case of $\mathcal{L}_2$ loss $\frac{1}{2}(f(\mathbf{Q}\mathbf{x}_i) - y_i)^2$, the chain rule (vectorized over all samples) results

$$\frac{\partial \mathcal{L}}{\partial Q_{r,c}} = ([f(\mathbf{Q}\mathbf{x})] - \mathbf{y})^\top \left[\frac{\partial f(\mathbf{Q}\mathbf{x})}{\partial Q_{r,c}} \right].$$

III. Dummy Head2

III. Remark 1 (Non-linear feature transforms): One can readily extend our linear transform to non-linear transforms. The derivations leading up to (2) rely only on the soft-decision-tree (SDT) structure; the specific form of the feature transform $\phi: \mathbb{R}^{d_{\text{in}}} \rightarrow \mathbb{R}^{d_{\text{out}}}$, $\mathbf{x} \mapsto \mathbf{z} = \phi_\Theta(\mathbf{x})$ does not appear. Consequently, we can replace the current linear map $\mathbf{Q}$ with any differentiable transform $\phi_\Theta(\cdot)$, such as NN's. Starting from (3) we need to change

$$\frac{\partial f(\mathbf{z})}{\partial \theta} = \sum_{k=1}^{d_{\text{out}}} \underbrace{\frac{\partial f(\mathbf{z})}{\partial z_k}}_{\text{depends on SDT}} \underbrace{\frac{\partial z_k}{\partial \theta}}_{\text{depends on } \phi_\Theta}, \quad (4)$$

where θ any parameter $\in \Theta$, and only $\partial z_k / \partial \theta$ depends on the chosen transform. For example, if we use multi-layer perceptron as ϕ_Θ , the explicit expression to place in (4) would be $\frac{\partial z_k}{\partial a_i^{(l)}} \frac{\partial a_i^{(l)}}{\partial w_{ij}^{(l)}} = \delta_i^{(l)} h_j^{(l-1)}$.

Here $w_{ij}^{(l)}$ denotes the weight that links neuron j in layer $l-1$ to neuron i in layer l . Vectors $\mathbf{a}^{(l)}$ and $\mathbf{h}^{(l)}$ refer to the pre-activation and post-activation signals of layer l , respectively. The single-output back-prop term is therefore $\delta_i^{(l)} = \partial z_k / \partial a_i^{(l)} = \sigma'(a_i^{(l)})$ only when $i = k$.

One could likewise plug in more expressive architectures such as LSTMs or CNNs in a similar manner. However, every additional layer increases the number of trainable parameters and, consequently, the risk of over-fitting, defeating the purpose of this letter.

III. Remark 2 (Use of HDT as base learners): The non-differentiable splits of hard decision trees prevent us from computing $\partial \mathcal{L} / \partial \theta$ for the feature transform ϕ_θ . To extend our framework to this case, we would have to do back-and-forth optimization between (i) fitting the tree on the current features and (ii) adjusting ϕ_θ with the tree frozen, then repeating the cycle. This alternating scheme increases training time and may settle on a configuration that is not jointly optimal for the tree and the transform as shown in our experiments.

IV. SIMULATIONS

A. Synthetic Data Analysis

To investigate the feature-selection capability of our method in a controlled setting, we construct a synthetic dataset that combines a static linear regression signal with a dynamic ARMA noise term, thus turning the task into a time-series (i.e., sequential regression) problem.

For each of the $S = 5$ univariate series and every time index $t \in \{1, \dots, 212\}$ we draw $d = 50$ exogenous covariates $x_{s,t,i} \sim \mathcal{N}(0, 1)$. Only the first $d_{\text{rel}} = 10$ covariates are truly informative; the remaining $d_{\text{irr}} = 40$ are redundant. We sample once—and then keep fixed across all series—independent weights $w_i \sim \mathcal{N}(0, 1)$ for $i = 1, \dots, d_{\text{rel}}$. The static part of the signal is generated with `make_regression` from `scikit-learn` [17], which also adds Gaussian noise with $\sigma = 0.3$. To inject temporal dependence we superimpose an ARMA(1, 1) process with parameters $\varphi = 0.85$ and $\theta = 0.75$:

$$\varepsilon_{s,t} = \varphi \varepsilon_{s,t-1} + u_{s,t} + \theta u_{s,t-1}, \quad u_{s,t} \sim \mathcal{N}(0, 0.5^2),$$

and obtain the target $y_{s,t} = \sum_{i=1}^{d_{\text{rel}}} w_i x_{s,t,i} + \lambda \varepsilon_{s,t}$, $\lambda = 200$.

B. Real-World Time Series

We next evaluate our algorithm over three well-known publicly-available, real-life datasets that represent diverse application domains and sampling frequencies:

- M4 (quarterly). M4 forecasting competition time series, drawn from domains such as demographics, finance, industry and tourism [18].
- Weather (10-minute). Twenty-one meteorological measurements from the Max-Planck Biogeochemistry station in Jena (calendar year 2020) [19].
- Exchange (daily). 8 national exchange rates against the USD observed between 1990 – 2016, [20].
- ETTm2 (15-minute). Electricity transformer data collected in Eastern China (July 2016 – July 2018) [21].

Our preprocessing pipeline begins with the scaling of the target variable. This is an important step, as it ensures that both the histogram-based and neural network-based methods perform equally well [22]. Then, we generate lagging and rolling statistics as features. This procedure purposefully creates a data scarce, high dimensional setting that our algorithm is designed to handle. Note that choices of these features are arbitrary and the primary concern is to test the robustness of our feature transformation mechanism in a redundant feature-rich environment. Table I shows the dataset statistics.

C. Evaluation and Results

We compare our proposed BSDT-Q model against a comprehensive set of baseline methods spanning different

TABLE I
DESCRIPTIVE STATISTICS FOR EACH DATASET

Statistic	M4	Weather	Exchange	ETtm2	Synthetic
Features	17	23	36	20	53
Target Mean	0.1356	0.0338	-0.1967	-0.0154	-0.0440
Target Std	0.8483	0.0090	0.0354	0.0986	0.9890
Test Length	40	12	28	48	60
Train Length	262	200	100	100	1000

TABLE II
TEST MSE FOR EACH MODEL ACROSS DATASETS. (BEST RESULTS ON EACH DATASET ARE **BOLDED**.)

Model	M4	Weather	Exchange	ETtm2	Synthetic
BSDT-MLP	9.24e-1	8.27e-5	1.95e-3	1.01e-2	1.22e+0
BSDT-Q (Ours)	5.64e-1	6.90e-5	4.95e-4	4.34e-3	1.14e+0
Catboost	9.97e-1	1.22e-4	3.99e-3	6.41e-3	1.46e+0
LSTM	6.61e-1	1.54e-4	1.71e-3	5.85e-3	1.28e+0
Lasso	7.60e-1	2.81e-4	7.74e-3	1.11e-2	1.26e+0
LightGBM	8.48e-1	1.10e-4	8.23e-4	6.92e-3	1.71e+0
RandomForest	7.49e-1	9.88e-5	1.60e-3	8.87e-3	1.32e+0
SDT	9.02e-1	1.28e-4	5.91e-4	1.01e-2	1.26e+0
XGBoost	7.74e-1	1.00e-4	9.19e-4	1.06e-2	1.43e+0

TABLE III
TRAINING TIMES (IN SECONDS) FOR EACH MODEL ACROSS DATASETS. (ALL EXPERIMENTS ARE RUN ON A M3 MACBOOK PRO.)

Model	M4	Weather	Exchange	ETtm2	Synthetic
BSDT-MLP	0.06	0.67	0.08	0.28	1.15
BSDT-Q (Ours)	0.31	0.15	0.05	0.29	0.14
Catboost	1.74	0.18	0.78	0.11	2.79
LSTM	0.08	0.06	0.06	0.05	0.28
Lasso	0.01	0.01	0.02	0.04	0.02
LightGBM	0.12	0.38	0.38	0.15	1.94
RandomForest	0.07	0.08	0.04	0.05	0.15
SDT	0.11	0.04	0.04	0.10	0.31
XGBoost	0.03	0.04	0.06	0.07	0.22

algorithmic paradigms. As state-of-the-art decision tree boosting methods, we include LightGBM [4], XGBoost [3], and CatBoost [23]. We also evaluate classical machine learning approaches including Random Forest [24] and Lasso Regression [25]. To assess performance against neural network-based sequence models, we include an LSTM architecture [26]. For ablation studies, we compare against a single SDT [27] to demonstrate the benefits of boosting, and BSDT-MLP, which uses multi-layer perceptron feature transforms between boosting layers instead of our proposed Q-transform mechanism.

All models were tuned and fitted using the Optuna framework [28] with a TPE sampler [29], implemented on the nixtla/mlforecast library [30]. The best model was selected by searching over the hyperparameter spaces defined in Table IV, based on performance on a backtesting window, with an equal optimization budget allocated to each model. For the full details of the experimental procedure, please refer to the reproducibility package.

Table II demonstrates that our algorithm significantly outperforms all baseline models across every dataset. Moreover, Table III reveals that our approach is computationally efficient, with training times comparable to or lower than those

TABLE IV
HYPERPARAMETER SEARCH SPACE FOR EACH MODEL

Model	Parameter	Search Space
BSDT Variants	Boosting Layers	(1, 2, 4)
	Depth	(2, 4, 6)
	Epochs	(10, 20)
	Hidden Dimension	(None, 16, 32)
	L1-L2 Regularization	(0.01, 0.1, 0.2)
Catboost	Learning Rate	(0.005, 0.01, 0.05)
	Depth	(8, 10, 12)
	Estimators	(50, 100, 200)
	L1-L2 Regularization	(1, 0.1, 0.01)
	Learning Rate	(0.1, 0.3)
LSTM	Min. Child Samples	(1, 2)
	Dropout	(0.1, 0.2, 0.3)
	Epochs	(10, 20, 30)
	Hidden Size	(50, 100, 150)
	Learning Rate	(0.001, 0.005, 0.01)
Lasso	Num Layers	(1, 2, 3)
	Alpha	(0.0001, 0.001, 100.0, 1000.0)
LightGBM	Max. Iterations	(100, 200)
	Depth	(-1, 15, 20)
	Estimators	(50, 100, 200)
	L1-L2 Regularization	(0, 0.001)
	Learning Rate	(0.1, 0.3, 0.5)
RandomForest	Leaves	(127, 255, 511)
	Min. Child Samples	(1, 5, 10)
	Depth	(None, 20, 30)
XGBoost	Estimators	(10, 50, 100)
	Max. Features	('sqrt', 'log2', 1.0)
	Min. Child Samples	(1, 2, 5)
	Min. Samples Split	(2, 5, 10)
XGBoost	Depth	(10, 15, 20)
	Estimators	(50, 100, 200)
	L1-L2 Regularization	(1, 0.1, 0)
	Learning Rate	(0.1, 0.3, 0.5)
	Min. Child Samples	(1, 2, 5)
XGBoost	Min. Split Loss	(0, 0.1)

of competing methods. Notably, the ablation studies validate the effectiveness of our proposed Q-transform mechanism: BSDT-Q outperforms both the single SDT baseline and BSDT-MLP on all datasets.

V. CONCLUSION

In this letter, we introduce a soft gradient boosting framework for sequential regression that integrates a learnable linear feature transform into boosting. We fit a soft decision tree on the current inputs and then learn a transform matrix Q in an end-to-end manner, allowing feature selection and model fitting together at each boosting iteration. We also show how the same approach can be extended to other differentiable transforms and hard decision trees. Through experiments on both synthetic and real-world datasets, we show that our algorithm notably increases the performance of boosted trees while adding minimal computational overhead in data-scarce, high-dimensional settings. To support reproducibility and further work, we make our implementation publicly available.

REFERENCES

- [1] Y. Zhang, "Hourly traffic forecasts using interacting multiple model (IMM) predictor," *IEEE Signal Process. Lett.*, vol. 18, no. 10, pp. 607–610, Oct. 2011. [Online]. Available: <https://doi.org/10.1109/LSP.2011.2165537>
- [2] X. Song, Y. Guo, N. Li, and L. Zhang, "Integrated online prediction model for iot data," *IEEE Signal Process. Lett.*, vol. 28, pp. 2043–2047, 2021. [Online]. Available: <https://doi.org/10.1109/LSP.2021.3115042>
- [3] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. of the 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, Aug. 2016, pp. 785–794. [Online]. Available: <http://doi.org/10.1145/2939672.2939785>
- [4] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30, pp. 3146–3154. [Online]. Available: <https://dl.acm.org/doi/10.5555/3294996.3295074>
- [5] S. Makridakis and F. Petropoulos, "The M4 competition: Conclusions," *Int. J. Forecasting*, vol. 36, no. 1, pp. 224–227, 2020. [Online]. Available: <https://doi.org/10.1016/j.ijforecast.2019.05.006>
- [6] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "M5 accuracy competition: Results, findings, and conclusions," *Int. J. Forecasting*, vol. 38, no. 4, pp. 1346–1364, 2022. [Online]. Available: <https://doi.org/10.1016/j.ijforecast.2021.11.013>
- [7] A. Hakami, "Strategies for overcoming data scarcity, imbalance, and feature selection challenges in machine learning models for predictive maintenance," *Sci. Rep.*, vol. 14, no. 1, 2024, Art. no. 9645. [Online]. Available: <https://doi.org/10.1038/s41598-024-59958-9>
- [8] V. Bansal, H. Buckchash, and B. Raman, "Discriminative auto-encoding for classification and representation learning problems," *IEEE Signal Process. Lett.*, vol. 28, pp. 987–991, 2021. [Online]. Available: <https://doi.org/10.1109/LSP.2021.3077853>
- [9] V. Bolón-Canedo, N. Sánchez-Marño, and A. Alonso-Betanzos, "Feature selection for high-dimensional data," *Prog. Artif. Intell.*, vol. 5, no. 2, pp. 65–75, 2016. [Online]. Available: <https://doi.org/10.1007/s13748-015-0080-y>
- [10] M. Kim, "Time-series dimensionality reduction via Granger causality," *IEEE Signal Process. Lett.*, vol. 19, no. 10, pp. 611–614, Oct. 2012. [Online]. Available: <https://doi.org/10.1109/LSP.2012.2209641>
- [11] J. Feng, Y.-X. Xu, Y. Jiang, and Z.-H. Zhou, "Soft gradient boosting machine," 2020. [Online]. Available: <https://doi.org/10.48550/arXiv.2006.04059>
- [12] X. Liu and T. Yu, "Gradient feature selection for online boosting," in *Proc. IEEE 11th Int. Conf. Comput. Vis.*, 2007, pp. 1–8. [Online]. Available: <https://doi.org/10.1109/ICCV.2007.4408912>
- [13] Z. Xu, G. Huang, K. Q. Weinberger, and A. X. Zheng, "Gradient boosted feature selection," in *Proc. of the 20th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2014, pp. 522–531. [Online]. Available: <https://doi.org/10.48550/arXiv.1901.04055>
- [14] E. Sarma, N. Dimitropoulos, V. Marinakis, Z. Mylona, and H. Doukas, "Transfer learning strategies for solar power forecasting under data scarcity," *Sci. Rep.*, vol. 12, no. 1, 2022, Art. no. 14643. [Online]. Available: <https://doi.org/10.1038/s41598-022-18516-x>
- [15] S. B. Taieb and R. J. Hyndman, "Recursive and direct multi-step forecasting: The best of both worlds," in *Proc. Dept. Econometrics Bus. Statist., Monash Univ.*, 2012, vol. 19. [Online]. Available: <https://doi.org/10.26180/21500169.v1>
- [16] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001. [Online]. Available: <https://doi.org/10.1214/aos/1013203451>
- [17] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://doi.org/10.48550/arXiv.1201.0490>
- [18] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M4 competition: 100,000 time series and 61 forecasting methods," *Int. J. Forecasting*, vol. 36, no. 1, pp. 54–74, 2020. [Online]. Available: <https://doi.org/10.1016/j.ijforecast.2019.04.014>
- [19] H. A. Wu, "Autoformer: Decomposition transformers with autocorrelation for long-term series forecasting," in *Proc. 35th Int. Conf. Neural Inf. Process. Syst.*, 2021, Art. no. 1717. [Online]. Available: <https://doi.org/10.48550/arXiv.2106.13008>
- [20] G. Lai, W. Chang, Y. Yang, and H. Liu, "Modeling long and short-term temporal patterns with deep neural networks," in *Proc. Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2018, pp. 95–104. [Online]. Available: <https://doi.org/10.48550/arXiv.1703.07015>
- [21] H. Zhou et al., "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. 35th AAAI Conf. Artif. Intell.*, 2021, pp. 11106–11115. [Online]. Available: <https://doi.org/10.48550/arXiv.2012.07436>
- [22] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016. [Online]. Available: <http://www.deeplearningbook.org>
- [23] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Proc. 32nd Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 6639–6649.
- [24] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001. [Online]. Available: <https://doi.org/10.1023/A:1010933404324>
- [25] R. Tibshirani, "Regression shrinkage and selection via the lasso," *J. Roy. Statist. Soc. Ser. B (Methodological)*, vol. 58, no. 1, pp. 267–288, 1996. [Online]. Available: <https://doi.org/10.1111/j.2517-6161.1996.tb02080.x>
- [26] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997. [Online]. Available: <https://doi.org/10.1162/neco.1997.9.8.1735>
- [27] H. Luo, F. Cheng, H. Yu, and Y. Yi, "SDTR: Soft decision tree regressor for tabular data," *IEEE Access*, vol. 9, pp. 55999–56011, 2021. [Online]. Available: <https://doi.org/10.1109/ACCESS.2021.3070575>
- [28] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 2623–2631. [Online]. Available: <https://doi.org/10.48550/arXiv.1907.10902>
- [29] S. Watanabe, "Tree-structured parzen estimator: Understanding its algorithm components and their roles for better empirical performance," 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2304.11127>
- [30] Nixtla, "mlforecast: Scalable machine learning based forecasting," 2021. [Online]. Available: <https://github.com/Nixtla/mlforecast>